

Luminar Iris Per-Point Timestamp

Root Cause and Validation Procedure for GLIM & GICP Deskew

AV-24 / DLIO++ mapping & localization stack | To be executed jointly against the committed code and live/bagged Luminar data.

Purpose. Deskewing in both pipelines (GLIM and gicp_localization) now depends on per-point Luminar timestamps. The decoders assume a specific bit-level layout that the sensor datasheet does *not* guarantee and that the ROS2 driver is responsible for producing. This document states the root cause precisely, enumerates the failure modes, and gives a concrete, reproducible procedure to confirm the real on-the-wire format and prove that deskew is correct for each pipeline. Work through it side-by-side with the committed diagnostics.

1. System context: where timestamps are decoded

Both pipelines read a per-point time field out of `sensor_msgs/PointCloud2` and convert it to a per-point capture time used to motion-compensate (deskew) the scan against IMU integration. The decode sites are:

Pipeline	Decode site (committed)	Assumed Luminar encoding	Conversion
GLIM	<code>glim/util/ros_cloud_converter.hpp</code> — <code>PointField UINT8, count==8</code> branch	little-endian <code>uint64</code> = PTP epoch nanoseconds	<code>times[i] = u64 / 1e9</code> → epoch seconds
GICP	<code>gicp_localization/src/localization.cc</code> — <code>copyPointTimeFromCloud()</code> , <code>SensorType::LUMINAR</code>	<code>UINT8[8]</code> or <code>FLOAT64</code> -wrapped <code>uint64</code> bits	stores raw <code>uint64</code> ns; deskew uses <code>(ts - min_ts) * 1e-9</code>

Downstream, GLIM's TimeKeeper auto-detects the encoding from the first cloud's min/max times; the divide-by-1e9 lands `min_time` near 1.7e9 (epoch seconds), so it takes the *absolute* → *relative* branch with `point_time_scale = 1.0`. GICP deskews on the intra-scan relative offset so the absolute epoch reference cancels. Both are correct **only if** the per-point value really is a full `uint64` epoch-nanosecond reconstruction.

2. Root cause

The Luminar Iris does not emit a single `uint64` epoch-nanosecond timestamp on the wire. Per the **Luminar Iris Data Output Specification v1.3.0**, the timestamp is split across two places:

- **Packet-header seconds** (§2.1): PTP Timestamp – seconds, `UQ48.0`, 48-bit integer epoch **seconds**, carried *once per packet*.
- **Per-ray nanoseconds** (§2.2 / §2.6.3): PTP Timestamp – nanoseconds, `UQ32.0`, 32-bit **sub-second** nanosecond count (0–10⁹), carried *per ray*; it wraps every 1 s.
- All fields are little-endian (§2). Packets are second-aligned: all rays from one second are transmitted before any ray from the next.

The single `uint64` epoch-ns value that GLIM and GICP read is therefore a **reconstruction performed by the ROS2 driver** (`header_seconds * 1e9 + ray_nanoseconds`), not a field the sensor publishes. The datasheet section the code comments previously cited for a packed `uint64` (“PIG R2.0.7 §7.8.4”) is actually *PTP Troubleshooting* — it does not describe the data layout. The Iris ROS2 driver's exact `PointField` layout is not publicly documented (it ships via the Luminar Portal), so the reconstruction behavior is currently **an assumption, not a verified fact**.

Root cause, in one sentence: correctness of GLIM/GICP deskew hinges on the ROS2 driver reconstructing the split header-seconds + per-ray-nanoseconds into a single little-endian `uint64` epoch-ns per point — and that has never been confirmed against real data.

3. Failure modes if the driver does something else

Driver actually publishes	What the decoder does	Symptom in deskew / mapping
Full epoch ns as uint64 (assumed)	Correct: span ~ scan period; epoch matches wall clock	Correct. Sharp deskewed scans.
Bare 32-bit sub-second ns only (no seconds added)	Reads a 0–10 ⁹ sawtooth as the absolute time	Deskew jumps / tears whenever a scan straddles a 1 s boundary (value wraps to ~0).
FLOAT64 seconds (true IEEE double), not packed bits	GICP/GLIM memcopy the bits as uint64 → scrambled	Garbage per-point times; deskew smears badly or is rejected.
Collapsed: every point = one scan-level stamp	Zero intra-scan span	No motion compensation; high-speed scans smear uniformly.
Different field name / datatype / count than expected	Time field not found, or wrong width read	Deskew silently disabled, or misread bytes.

At 80–200 mph these are not cosmetic: a 0.1 s scan covers up to ~9 m, so a mis-decoded per-point time directly bends or smears the scan and corrupts registration.

4. Validation instruments (already committed)

GICP — [LUMINAR_TS_DIAG] **block.** `logLuminarTimestampStats()` in `localization.cc` prints, to `stderr`, the full field schema and then interprets the 8 timestamp bytes of `point[0]`, `point[mid]`, `point[N-1]` four ways, with adjacent-sample deltas and a heuristic verdict. It is invoked from the scan path (raw scan and deskewed scan). This is the primary instrument — no code change needed to read the real format.

```
[LUMINAR_TS_DIAG] BEGIN
[LUMINAR_TS_DIAG] sensor_type=luminar point_step=.. ...
[LUMINAR_TS_DIAG] fields (offset, datatype, count, name):
[LUMINAR_TS_DIAG]   off=.. type=UINT8 count=8 name='t'           <- inspect this row
[LUMINAR_TS_DIAG] timestamp_field: off=.. type=.. count=..
[LUMINAR_TS_DIAG] point[0]   raw=ab cd .. as_uint64=.. as_double_sec=.. as_double_ns=..
[LUMINAR_TS_DIAG] deltas (mid-first / last-first):
[LUMINAR_TS_DIAG]   as_uint64_ns: mid-first=.. last-first=..   <- should be ~ +half / +full scan period
[LUMINAR_TS_DIAG] verdict (heuristic): uint64 ns interpretation looks plausible (span .. ms)
[LUMINAR_TS_DIAG] END
```

GLIM — **no built-in dump.** Validate GLIM by inspecting the topic/bag directly (Appendix B script) and by confirming TimeKeeper's branch selection and deskew output. The byte-level conclusions from the GICP TS_DIAG transfer directly to GLIM because both read the same driver output.

5. Test procedure A — identify the real field schema

Goal: record what the driver actually publishes for the time field. Do this on the live sensor and on a representative bag.

- 1 Echo one message's structure (no array spam): `ros2 topic echo --no-arr /luminar_front/points`. Record `fields[]` (name, offset, datatype, count), `point_step`, `is_bigendian`, `height`, `width`.
- 2 Confirm a time field exists with a recognized name: `t`, `time`, `time_stamp`, or `timestamp` (this is what both pipelines search for). Record its **datatype** and **count**.
- 3 Map datatype: 7=FLOAT32, 8=FLOAT64, 6=UINT32, 2=UINT8. The assumed Luminar layout is UINT8 with count=8 (an 8-byte run) or a FLOAT64 wrapper. Anything else is a finding.
- 4 For GICP, run the node with `localization/sensor_type: luminar` and capture the `[LUMINAR_TS_DIAG]` block from `stderr` for the same cloud.

6. Test procedure B — magnitude & intra-scan span

Goal: decide which interpretation is real from the numbers. Use the `TS_DIAG` as_uint64 / as_double_sec columns, or the Appendix B script.

Observation on the time bytes	Interpretation	Verdict
as_uint64 ~ 1.7e18, and last-first ~ 1e8 (=0.1 s in ns)	Full epoch nanoseconds, per-point	PASS — matches the decoder assumption.
as_uint64 in 0 .. 1e9, sawtooth, sometimes last-first < 0	Bare sub-second ns (seconds dropped)	FAIL — will wrap at 1 s boundaries.
as_uint64 ~ 1.7e9 (looks like seconds), span tiny	Epoch seconds , not ns	FAIL for GICP (expects ns); GLIM /1e9 would be wrong too.
as_double_sec ~ 1.7e9 and plausible; as_uint64 nonsense	True IEEE FLOAT64 seconds	FAIL — bytes must be read as double, not uint64.
Both deltas ~ 0 across all points	Collapsed scan-level stamp	FAIL — no per-point info; deskew ineffective.

Expected-good signature at 10 Hz: a per-point uint64 near 1.7e18 ns, monotonic non-decreasing within a detector, and a first-to-last span approximately equal to the scan period (~0.1 s = 1.0e8 ns).

7. Test procedure C — the one-second boundary test (critical)

This is the single most important test, because it is the only one that distinguishes a correct full-epoch reconstruction from a bare sub-second field that *looks* fine within a single scan.

- 1 Record a bag of ≥ 60 s of continuous Luminar data while the vehicle moves (or with the sensor running steadily).
- 2 Scan the per-point times for a cloud whose rays straddle a UTC second rollover (sub-second part crosses from ~0.999e9 back toward 0). The Appendix B script flags these automatically.
- 3 **Pass**: reconstructed per-point times remain monotonic across the rollover (the seconds term increments, so the absolute value keeps rising). **Fail**: a backward jump of ~1e9 ns appears mid-scan — proof the driver forwarded only the 32-bit sub-second field.
- 4 Repeat at speed: confirm the deskewed scan shows no discontinuity or doubled geometry at the moment of a second rollover.

8. Test procedure D — epoch / PTP wall-clock sanity

- 1 Take one point's reconstructed epoch (uint64 ns / 1e9) and convert to UTC. It must match wall-clock time to within PTP sync error (sub-millisecond when locked).
- 2 Confirm Atlas/PTP is FIXED/locked during capture (heading + time sync). If the sensor is unsynced, timestamps are time-since-boot, not epoch — deskew still works on relative offsets, but the absolute-epoch branch logic and any GT time-matching will be wrong.
- 3 Note the offset sign vs TAI/UTC if precise cross-sensor association is needed; deskew itself is offset-invariant (uses differences).

9. GLIM-specific validation

- 1 Confirm config: `autoconf_perpoint_times: true, autoconf_prefer_frame_time: false` in `config_sensors.json`.
- 2 Add a temporary log (or use a debugger/Appendix B) to confirm the `UINT8 count==8` branch in `ros_cloud_converter.hpp` is taken and that `times[i] = u64/1e9` produces values near $1.7e9$ (epoch seconds).
- 3 Verify TimeKeeper branch: `min_time ~ 1.7e9` → *absolute, auto-convert to relative*; `point_time_scale` stays 1.0 (NOT the $>1e16$ raw-ns branch). Log `point_time_scale` if unsure.
- 4 Functional deskew check: map a known straight, high-speed segment. With deskew correct, building edges/poles render single and sharp; with bad per-point times they smear or double along the travel direction.
- 5 Regression guard: temporarily set `autoconf_prefer_frame_time: true` (collapses to frame stamp = deskew off) and confirm the map visibly degrades — proves per-point times are actually being used.

10. GICP-specific validation

- 1 Set `localization/sensor_type: luminar` and `dllo/deskew: true`.
- 2 Read the `[LUMINAR_TS_DIAG]` verdict for both the raw scan and the deskewed scan; both should report the `uint64 ns` interpretation plausible with a ~ 0.1 s span.
- 3 Confirm `copyPointTimeFromCloud()` takes the LUMINAR branch and stores the raw `uint64` bits, and that `deskewPointcloud()` computes $(ts - min_ts) * 1e-9$ with `min_ts` in epoch ns (log `min_ts`; expect $\sim 1.7e18$).
- 4 Check the per-point offsets span the scan (0 .. ~ 0.1 s) and are non-decreasing within a detector (the spec allows cross-detector disorder — do not assume global sort).
- 5 Functional check at speed: confirm GICP fitness improves with deskew on vs off, and that the jump gate is not firing because of time-induced smearing.

11. Auxiliary LiDAR concatenation validation

The front + left + right Luminars are concatenated (`GICP mergeAuxClouds()`; `GLIM merge_clouds()`). Both now enforce full `PointCloud2` schema equality before appending, and both treat `UINT8[8]` as absolute epoch and do **not** shift it during the merge.

- 1 Confirm all three topics publish an identical field schema (same name/offset/datatype/count/point_step). If the schema gate logs a skip, the aux layout differs — that is a finding to resolve upstream.
- 2 After merge, run the one-second boundary test (Procedure C) on the combined cloud: aux points carry their own absolute epoch and must remain coherent with the primary after deskew.
- 3 Verify no double-shift: since `UINT8[8]` is absolute, the per-point times of aux rays should equal their true capture epoch (not rebased by the inter-scan dt).

12. Acceptance criteria

#	Criterion	Met when
1	Field schema identified	datatype/count/offset recorded for the time field on live + bag, all three Luminars.
2	Encoding confirmed	TS_DIAG / script shows full epoch-ns uint64 (span ~ scan period), not sub-second / double / collapsed.
3	Second-boundary safe	Procedure C: no backward jump across a 1 s rollover, in single and merged clouds.
4	Epoch sane	A point's epoch matches wall clock within PTP error; PTP locked during capture.
5	GLIM branch correct	UINT8[8] /1e9 path taken; TimeKeeper absolute→relative; point_time_scale=1.0.
6	GICP path correct	LUMINAR decode + (ts-min_ts)*1e-9; min_ts ~ 1.7e18; offsets span scan.
7	Functional deskew	High-speed scan sharpens with deskew on; degrades with per-point times disabled.

13. If validation fails — remediation map

Finding	Action
Bare sub-second ns (Procedure C fails)	Reconstruct epoch in the driver, or add header-seconds + ray-ns combination at the decode site before /1e9; re-test boundary.
True FLOAT64 seconds	Read the field as double (not uint64 bits) in both decoders; GICP already has a FLOAT64 path — route Luminar to it and skip the uint64 memcpy.
Collapsed scan-level stamp	Driver/firmware config: enable per-ray timestamps; until then keep deskew off (frame-time) to avoid false motion comp.
Schema differs across aux topics	Fix the publisher/driver config so all three match; the schema gate will keep mismatched aux out until then.
Unsynced clock (epoch wrong)	Restore PTP lock; deskew still works on relative offsets but fix before trusting absolute/GT time association.

Appendix A — commands

```
# 1) Field schema for one message (no giant arrays)
ros2 topic echo --no-arr /luminar_front/points | head -n 60

# 2) Rate / continuity (look for drops)
ros2 topic hz /luminar_front/points

# 3) Record a >=60 s bag spanning a UTC second rollover, all three LiDARs
ros2 bag record -o luminar_ts_check \
  /luminar_front/points /luminar_left/points /luminar_right/points

# 4) GICP node with the diagnostic; capture the TS_DIAG block from stderr
ros2 run gicp_localization localization_node \
  --ros-args -p localization/sensor_type:=luminar 2> ts_diag.log
grep LUMINAR_TS_DIAG ts_diag.log | head -n 40
```

Appendix B — standalone PointCloud2 timestamp inspector

Reads a bag (or live topic via a trivial change) and reports the time-field schema, the four byte interpretations, intra-scan span, and — crucially — flags any backward jump indicative of a bare sub-second field. Use this for GLIM (which has no built-in dump) and as an independent cross-check for GICP.

```
import sys, struct
from rosbags.highlevel import AnyReader # pip install rosbags
from pathlib import Path

DT = {1:'INT8',2:'UINT8',3:'INT16',4:'UINT16',5:'INT32',6:'UINT32',7:'FLOAT32',8:'FLOAT64'}
TOPIC = '/luminar_front/points'

def find_time(fields):
    for f in fields:
        if f.name in ('t','time','time_stamp','timestamp'):
            return f
    return None

with AnyReader([Path(sys.argv[1])]) as r:
    conns = [c for c in r.connections if c.topic == TOPIC]
    for i,(conn,_,raw) in enumerate(r.messages(connections=conns)):
        m = r.deserialize(raw, conn.msgtype)
        tf = find_time(m.fields)
        if tf is None:
            print('NO TIME FIELD; fields=', [f.name for f in m.fields]); break
        if i == 0:
            print('time field: name=%s datatype=%s count=%d offset=%d point_step=%d big_endian=%d'
                  % (tf.name, DT.get(tf.datatype,tf.datatype), tf.count, tf.offset,
                     m.point_step, m.is_bigendian))
        n = m.width * m.height
        data = bytes(m.data)
        def u64(k):
            o = k*m.point_step + tf.offset
            return struct.unpack_from('<Q', data, o)[0]
        def f64(k):
            o = k*m.point_step + tf.offset
            return struct.unpack_from('<d', data, o)[0]
        idx = [0, n//2, n-1]
        u = [u64(k) for k in idx]
        print('cloud %d n=%d as_uint64=%s span_ns=%d as_double_sec[0]=%.6f'
              % (i, n, u, u[-1]-u[0], f64(0)))
        # one-second boundary / monotonicity scan over all points
        prev, bad = None, 0
        for k in range(n):
            v = u64(k)
            if prev is not None and v + 1_000_000 < prev: # >1 ms backward = suspicious
                bad += 1
            prev = v
        if bad:
            print('    !! %d backward jumps (sub-second wrap?) -> Procedure C FAIL' % bad)
```

```

if i >= 50:
    break

```

Run: `python3 ts_inspect.py luminar_ts_check`. A clean result prints uint64 values near 1.7e18, span_ns near 1e8 at 10 Hz, and zero backward jumps. Any backward-jump report is the bare-sub-second failure.

Appendix C — code reference map

Concern	File : symbol
GLIM decode (UINT8[8] /1e9)	GLIM/glim/include/glim/util/ros_cloud_converter.hpp : PointField UINT8 count==8
GLIM autoconf / branch	GLIM/glim/config/config_sensors.json : autoconf_perpoint_times, autoconf_prefer_frame_time
GLIM aux merge (no-shift UINT8[8])	GLIM/glim_ros2/include/glim_ros/lidar_concat.hpp : merge_clouds / shift_cloud_timestamps
GICP decode	gicp_localization/src/localization.cc : copyPointTimeFromCloud (SensorType::LUMINAR)
GICP diagnostic	gicp_localization/src/localization.cc : logLuminarTimestampStats ([LUMINAR_TS_DIAG])
GICP deskew	gicp_localization/src/localization.cc : deskewPointcloud (LUMINAR branch, (ts-min_ts)*1e-9)
GICP aux merge + schema gate	gicp_localization/src/localization.cc : mergeAuxClouds / auxSchemaMatchesPrimary

How to use this document: run Procedures A–D once on a representative bag, record the findings against the Section 12 checklist, then keep the Appendix B script in the repo as a CI/pre-deployment smoke test for any driver or firmware update. Treat Procedure C as the gate that must pass before trusting deskew at racing speed.